



Stored Procedures and UDFs with Borland® JDataStore™

**Increase application capability and get
encapsulation of business logic**

by Jens Ole Lauridsen

Borland Software Corporation

August 2002

Contents

Introduction	1
Creating a stored procedure	2
How can stored procedures have output parameters?	6
Expand the SQL for JDataStore™ database	8
Performance considerations	9
Conclusion	9

Introduction

Borland® JDataStore™ is a SQL database written entirely in Java™ which helps developers to write truly portable embedded, mobile, and Web-based applications. The JDataStore database supports the use of stored procedures to encapsulate business logic in the schema of a database and UDF (User Defined Function) to allow developers to extend the built-in SQL support. Where many other database vendors have invented their own SQL like language for stored procedures, the JDataStore database uses Java as the language of choice. The advantage is that no new programming skills are required. Stored procedures can also increase the performance of an application because they are executed on the database server, thereby decreasing the network traffic for a given operation.

What are stored procedures?

Stored procedures are code snippets, which are stored on the database server and executed from a SQL client. Usually an application developer or a database administrator writes a stored procedure to hide the complexity of a database schema. Often the stored procedure will execute several SQL queries against the tables of the database to yield the effect it was designed for. In the JDataStore database, these SQL queries are written in Java using the familiar JDBC® API. The desired effect may be to update a set of tables, to calculate an accumulated value from one or more tables, or add specialized integrity constraints. A stored procedure may have several parameters, which can be either input, output, or both.

JDataStore™

white paper
white paper

Example:

Consider an `ADD_ORDER` procedure, which takes a `customerId`, an `itemId`, and a `quantity` as input, and adds a record to the `ORDERS` table. However in addition, we would like to verify that this customer has paid for previous orders. To achieve this, the procedure can be written to throw an exception if this is not the case. This stored procedure is executed by running a SQL `CALL` statement like:

```
"CALL ADD_ORDER(?, ?, ?)"
```

The question marks indicate parameter markers.

What is a UDF?

A UDF, or User Defined Function, is a code snippet that is written to extend the built-in SQL support. Like stored procedures, they are executed on the database server and called from a SQL client. A UDF must return a value and is usually written by an application developer to be used in the `WHERE` clause of `SELECT` queries. However a UDF may also be called by itself similar to a stored procedure.

Example:

Consider a `MAX_VALUE` function, which takes two values: `value1` and `value2` and returns the largest of the two values. The UDF can be executed in a SQL statement like:

```
"SELECT * FROM PEOPLE WHERE MAX_VALUE(HEIGHT, 5*WIDTH) < ?"
```

or in a SQL `CALL` statement like:

```
"?=CALL MAX_VALUE(?, ?)"
```

The first question mark is an output parameter marker for the result of the function.

Creating a stored procedure for the JDataStore™ database

There are three steps involved in creating a stored procedure for the JDataStore database:

- Write the Java code for the stored procedure.
- Add the class to the classpath of the JDataStore server process.
- Create the binding of a SQL identifier to the Java method.

Here is an instance of all these steps, using the example `ADD_ORDER` from earlier. Let us assume that the schema looks something like this:

CUSTOMER TABLE		
CUST_ID	INT	
CREDIT	DECIMAL (10,2)	The credit a customer has.
NAME	VARCHAR (80)	

ORDERS TABLE		
CUST_ID	INT	
ITEM_ID	INT	
QUANTITY	INT	How many items
SALE_AMOUNT	DECIMAL (10,2)	Total sale amount
PAID	DECIMAL (10,2)	Amount paid so far

ITEMS TABLE		
ITEM_ID	INT	
NAME	VARCHAR (60)	Item name
PRICE	DECIMAL (10,2)	Unit price
STOCK	INT	Stock count

1. Writing the Java™ code:

In your favorite Java IDE, create a new class with the code in this example:

```
package com.mycompany.util;
import java.sql.*;
import java.math.BigDecimal;
public class MyClass {
    public static void addOrder(Connection con, int custId, int itemId,
                               int quantity) throws SQLException
    {
        String findItem =
            "SELECT PRICE*?, STOCK INTO ?, ? FROM ITEMS WHERE ITEM_ID=?";
        CallableStatement stmt = con.prepareCall(findItem);
        stmt.setInt(1, quantity);
        stmt.registerOutParameter(2, Types.DECIMAL, 2);
        stmt.registerOutParameter(3, Types.INTEGER);
        stmt.setInt(4, itemId);
```

```

stmt.executeUpdate();
BigDecimal amount = stmt.getBigDecimal(2);
int stock = stmt.getInt(3);
stmt.close();
if (stock < quantity)
    throw new SQLException("We only have "+stock+" left in stock");
String findOwed =
    "SELECT SUM(SALE_AMOUNT-PAID) INTO ? FROM ORDERS WHERE CUST_ID=?";
stmt = con.prepareStatement(findOwed);
stmt.registerOutParameter(1, Types.DECIMAL);
stmt.setInt(2, custId);
stmt.executeUpdate();
BigDecimal owed = stmt.getBigDecimal(1);
stmt.close();
owed = owed == null ? amount : owed.add(amount);
String findCredit =
    "SELECT CREDIT INTO ? FROM CUSTOMER WHERE CUST_ID=?";
stmt = con.prepareStatement(findCredit);
stmt.registerOutParameter(1, Types.DECIMAL);
stmt.setInt(2, custId);
stmt.executeUpdate();
BigDecimal credit = stmt.getBigDecimal(1);
stmt.close();
if (owed.compareTo(credit) > 0)
    throw new SQLException("Customer doesn't have that much credit");
String updateStock =
    "UPDATE ITEMS SET STOCK=STOCK-? WHERE ITEM_ID=?";
PreparedStatement prep = con.prepareStatement(updateStock);
prep.setInt(1, quantity);
prep.setInt(2, itemId);
prep.executeUpdate();
prep.close();
String insertOrder =
    "INSERT INTO ORDERS (CUST_ID, ITEM_ID, QUANTITY, SALE_AMOUNT)"+
        "VALUES (?, ?, ?, ?)";
prep = con.prepareStatement(insertOrder);

```

```

        prep.setInt(1, custId);
        prep.setInt(2, itemId);
        prep.setInt(3, quantity);
        prep.setBigDecimal(4, amount);
        prep.executeUpdate();
        prep.close();
    }
}

```

2. Add the class to the classpath of the JDataStore™ server process

After writing the code for the stored procedure and making sure it compiles, you will need to add this class file to the classpath of the JDataStore server process. If the shortcut for "JDataStore Server" is used, the JdsServer.config file in the JDataStore bin directory must be edited to add a line starting with the key "addpath." Look for other similar entries. The line may consist of either a directory path or a JAR file path. For example:

```
addpath c:/mydir/myjar.jar
```

3. Create the binding of a SQL identifier to the Java™ method

Now, that the Java code is ready to be executed, the JDataStore database needs to know about the method that can be called from SQL. To do this, a "CREATE JAVA_METHOD" statement must be issued like:

```
CREATE JAVA_METHOD ADD_ORDER AS 'com.mycompany.util.MyClass.addOrder';
```

Note, that the fully classified method name must be given as a SQL string.

Now the ADD_ORDER is ready to be called.

Example

After creating the stored procedure ADD_ORDER, here is a code snippet with an example of its use:

```

import java.sql.*;
class Something {
    // Assume:
    //   con:      is a valid connection to a JDataStore server with MyClass on the classpath.
    //   custId:   is the customer we are dealing with (validated before this call).
    //   itemId:   is the item the customer is ordering.
    //   quantity: is the number of this item ordered.
    //
    void addOrder(Connection con, int custId, int itemId, int quantity) {
        // We are using a PreparedStatement since there are input parameters, but no output parameters
        PreparedStatement stmt = con.prepareStatement("CALL ADD_ORDER(?,?,?);");
        stmt.setInt(1, custId);
        stmt.setInt(2, itemId);
        stmt.setInt(3, quantity);
        stmt.executeUpdate();
    }
}

```

When `Something.addOrder` is called in the client application, it turns around and calls the stored procedure `ADD_ORDER`, which causes `MyClass.addOrder` to be executed in the JDataStore server process. By making `MyClass.addOrder` into a stored procedure, only one statement has to be executed over a remote connection. The five statements executed by `MyClass.addOrder` are executed in process with the JDataStore server using a local connection.

Note, that no connection instance was passed to the call of the stored procedure `ADD_ORDER`. Only the actual logical parameters are passed. The JDataStore database will generate an implicit connection object, when it finds a stored procedure or UDF, where the first argument is expected to be a `java.sql.Connection` instance.

How can stored procedures have output parameters?

The Java language does not pass parameters by reference, so how can JDataStore stored procedures have output parameters? The JDataStore database will recognize array types from the parameter list of a stored procedure, and assume that the parameter is an output parameter of the element type. An array of length 1 will be passed to the Java method, and the method may place the output into the first element of the array. If any value was assigned to the input parameter before the call, that value will be passed as the first element in the array.

Some examples:

```
package com.mycompany.util;

public class MyClass {
    // INOUT: Add 5 to an integer value, ignore any NULL values
    //
    public static void addFive(int[] param) {
        param[0] += 5;
    }
    // INOUT: Add 4 to an integer value, keep NULL values NULL
    //
    public static void addFour(Integer[] param) {
        if (param[0] != null) {
            param[0] = new Integer(param[0].intValue()+4);
        }
    }
}
```

And their use:

```
import java.sql.*;

class SomeUse {
    void tryAdding(Connection con) throws SQLException {
        CallableStatement addFive = con.prepareCall("CALL ADD_FIVE(?)");
        addFive.setInt(1,17);
        addFive.registerOutParameter(1, Types.INTEGER);
        addFive.executeUpdate();
        int result5 = addFive.getInt(1);
        if (result5 != 22)
            throw new RuntimeException("Wrong result");
        addFive.close();
        CallableStatement addFour = con.prepareCall("CALL ADD_FOUR(?)");
        addFour.setInt(1,17);
        addFour.registerOutParameter(1, Types.INTEGER);
        addFour.executeUpdate();
        int result4 = addFour.getInt(1);
        if (result4 != 21)
```

```

        throw new RuntimeException("Wrong result");
    addFour.setNull(1, Types.INTEGER);
    addFour.executeUpdate();
    addFour.getInt(1);
    boolean wasNull4 = addFour.wasNull();
    if (!wasNull4)
        throw new RuntimeException("Wrong result");
    addFour.close();
}
}

```

Notice, that the implementation for `addFour` uses the `Integer` wrapper class for an integer. That allows the developer of `addFour` to recognize NULLs passed by the JDataStore database, and to set an output parameter to NULL to be recognized by JDataStore. In contrast, note that in the implementation for `addFive`, it is impossible to know if a parameter was NULL, and it is impossible to set the result of the output parameter to NULL.

Expand the SQL for JDataStore™ database

If for some reason, an operator (for example: a bit wise AND operator) is needed for a where clause, and the JDataStore database does not offer that operator, it can simply be written in Java and called as a UDF.

Use this ability with caution; since the JDataStore database doesn't know what this function does, JDataStore will not be able to use any indices to speed up this part of the query.

Consider the UDF example given earlier involving the `MAX_VALUE` UDF:

```
"SELECT * FROM PEOPLE WHERE MAX_VALUE(HEIGHT, 5*WIDTH) < ?"
```

This query is equivalent to the following query:

```
"SELECT * FROM PEOPLE WHERE HEIGHT < ? AND 5*WIDTH < ?"
```

Where the same value is given both parameter markers. This SQL statement yields the same result because we know how `MAX_VALUE` is implemented. However the JDataStore database will only be able to use any indices available for the `HEIGHT` and `WIDTH` column for the second query. If there were no such indices, the performance of the two queries would be about the same. The advantage of writing a UDF comes when functionality doesn't exist in JDataStore SQL in the first place (for example: a bit wise AND operator).

Performance considerations

Stored procedures are executed in the same Java Virtual Machine as the JDataStore database engine itself. This allows for execution with minimal overhead. While a stored procedure is executing SQL statements, no network traffic is generated. The stored procedure will have a local JDBC® connection, which has the same performance advantage of using the local JDBC driver for the JDataStore database instead of the remote driver.

Conclusion

Here are some benefits in using stored procedures and UDFs with the JDataStore database:

- Isolation of business logic such as integrity constraints in the database engine, which makes this logic available and reinforced for all clients.
- Data is retrieved locally, which is faster than sending that data to and from the client.
- JDataStore SQL language can be extended with Java functions.
- No performance penalty because the stored procedures are executing in the same virtual machine as the database itself.

Borland®

100 Enterprise Way
Scotts Valley, CA 95066-3249
www.borland.com | 831-431-1000

Made in Borland®. Copyright © 2002 Borland Software Corporation. All rights reserved. All Borland brand and product names are trademarks or registered trademarks of Borland Software Corporation in the United States and other countries. Java is a trademark or registered trademark of Sun Microsystems, Inc. in the U.S. and other countries. All other marks are the property of their respective owners. Corporate Headquarters: 100 Enterprise Way, Scotts Valley, CA 95066-3249 • 831-431-1000 • www.borland.com • Offices in: Australia, Brazil, Canada, China, Czech Republic, France, Germany, Hong Kong, Hungary, India, Ireland, Italy, Japan, Korea, the Netherlands, New Zealand, Russia, Singapore, Spain, Sweden, Taiwan, the United Kingdom, and the United States. •13396